Docking and scoring notebook

Protocol for docking ligands into a target protein and rescoring the poses:

1. **RDKit** generates a diverse, low-energy set of conformations for each ligand.
2. The binding site is defined either from a **protein–ligand complex PDB** (the ligand is extracted for you) or from **prepared files** (protein without ligand + extracted ligand).
3. **smina** (https://sourceforge.net/projects/smina/) docks the conformers into the site (rigid or flexible side chains).
4. **AEV-PLIG** (Valsson *et al.*, *Commun. Chem.* 2025) predicts the binding affinity (pK) of every pose.

The example is the SARS-CoV-2 main protease (Mpro) fragment structure 5RHD.

Kernel: `docking_scoring` conda env. One-off setup, in a terminal in this folder (see README.md for details and for installing smina):

```
conda env create -f environment.yml
conda run -n docking_scoring python -m ipykernel install --user --name docking_scoring --display-name
"Python (docking_scoring)"
conda run -n docking_scoring python aev_plig/download_models.py
```

Output files are written next to the input files they come from; all prediction results go to a `predictions` folder next to the docked SDF.

Widgets: each step has a panel with a **Run** button. Set `AUTO_RUN = True` in the next cell to run every step with the default settings when executing all cells.

In [1]:
```
import os
from pathlib import Path

import pandas as pd
from IPython.display import HTML, display
from rdkit import Chem
from rdkit.Chem import Draw

import docking_tools as dt

%matplotlib inline

AUTO_RUN = os.environ.get("DOCKING_AUTORUN") == "1"   # set True to run all steps with defaults
dt.PROTEIN_COLOURING = "rasmol"   # protein cartoons: "rasmol" (secondary structure), "rainbow" or "grey"

print(dt.cpu_note())
dt.library_versions()
```

```
24 cores available (16 performance + 8 efficiency)
```

Out[1]:

	old env (docking)	this env
python	3.8.19	3.12.14
rdkit	2024.3.5	2026.09.1
torch	2.4.1	2.10.0
torch_geometric	2.6.1	2.8.0.post1
torchani	2.2.4	2.9.0
py3Dmol	2.5.4	2.5.4
pandas	2.0.3	3.0.6
numpy	1.24.4	2.5.3
scipy	1.10.1	1.18.1
sklearn	1.3.2	1.9.1
biopandas	0.5.1	0.5.2
qcelemental	0.30.1	0.51.2
ipywidgets	8.1.5	8.1.9
matplotlib	3.7.3	3.11.2
pymol	-	3.1.0
openbabel	-	3.2.1

1. Ligands to dock

Set the path of the SDF file of structures you want to dock. The name must be on the first line of each record, e.g.

```
 AEM 10028511
MOE2019           2D
```

Conformers, docked poses and predictions will be written to the **same folder** as this file.

In [2]:
```
LIGAND_SDF = Path("example/Fordocking.sdf").resolve()   # input structures to dock

input_mols = dt.load_sdf(LIGAND_SDF)   # warns about unreadable or unnamed records
print(f"{len(input_mols)} molecules read from {LIGAND_SDF}")
```

```
7 molecules read from example/Fordocking.sdf
```

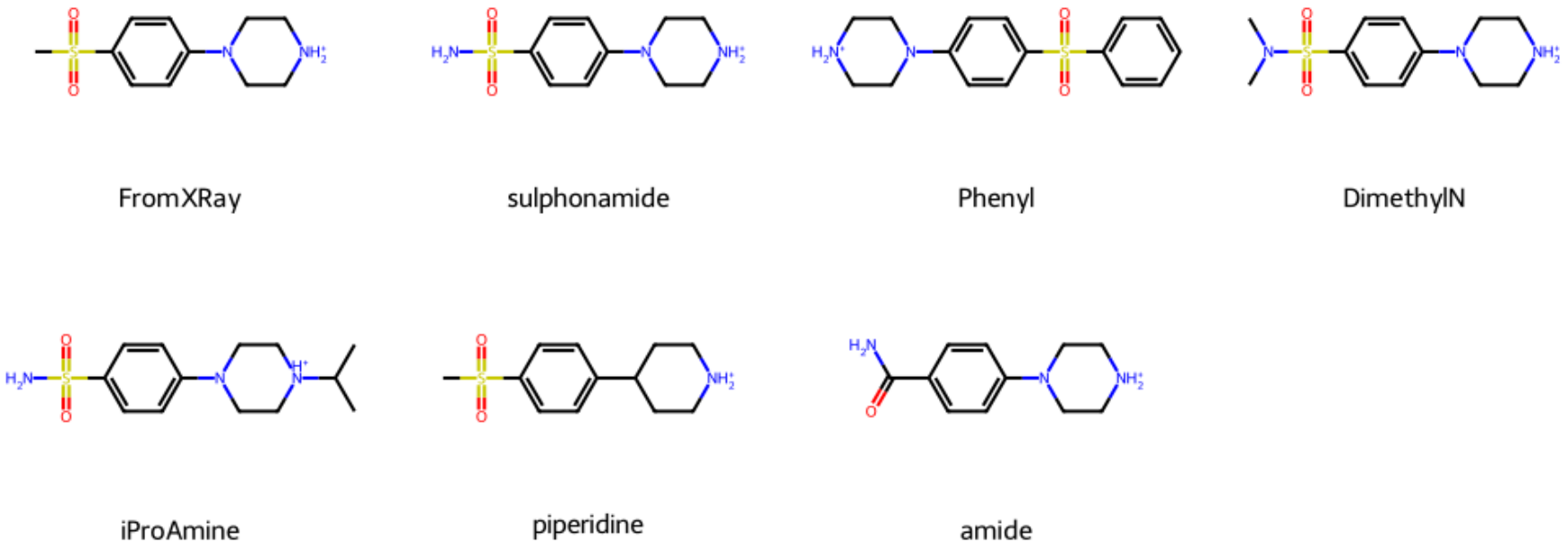
Check ligand names

smina keeps only the molecule name, and the results (best pose per ligand, PDB export, plots) are grouped by name, so **different structures must not share a name**. If duplicates are found you can either use an **identifier field** from the SDF (e.g. `ID`, `IDNUMBER`, `Record_ID` – offered only if every molecule has a unique value) as the ligand ID, or have **unique names assigned automatically** (`name`, `name_2`, `name_3` …). The original name is kept in the `original_name` property. The input SDF itself is not changed. Conformer generation is blocked until duplicates are resolved.

In [3]:
```
name_check = dt.LigandNamePanel(input_mols)
display(name_check.widget())
if AUTO_RUN and not name_check.resolved:
    name_check.apply()   # default: first unique identifier field, otherwise automatic names
```

In [4]:
```
# View as grid (re-run after applying a naming choice above)
Draw.MolsToGridImage([dt.to_2d(m) for m in input_mols], molsPerRow=4, subImgSize=(250, 200),
                     legends=[m.GetProp("_Name") for m in input_mols])
```

Out[4]:

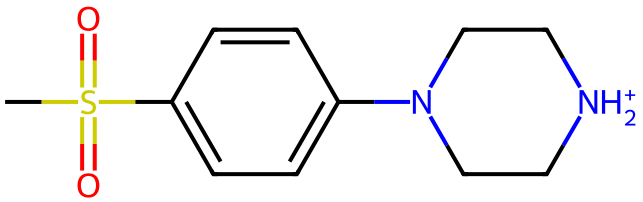


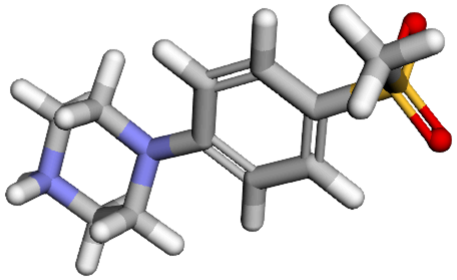
Browse individual input structures

Use the slider (or Prev / Next) to step through the molecules.

In [5]:
```
dt.molecule_browser(input_mols)
```

```
1/7  FromXRay   heavy atoms: 16   SMILES: CS(=O)(=O)c1ccc(N2CC[NH2+]CC2)cc1
```





2. Conformation generation

From the RDKit documentation: conformer generation is a difficult and subtle task. smina rotates around torsions during docking, but it does not flip rings or change amide geometry, so supplying several ring/amide conformers helps.

Are there better alternatives within RDKit than the original cell? Yes. The original notebook called `EmbedMultipleConfs(m, numConfs=3)` with no pruning or minimisation, so it could return near-duplicate, unrelaxed geometries. This notebook uses:

Step	What it does	Why
srETKDGv3 (default) / ETKDGv3 / ETKDG / ETDG	distance geometry + experimental torsion preferences	v3 has improved torsion libraries and macrocycle handling; the *sr* variant adds small-ring torsion preferences (better 3–7 membered ring puckers)
`pruneRmsThresh`	discards embeddings closer than the RMS threshold	diversity rather than duplicates
MMFF94 optimisation (UFF fallback)	relaxes bond lengths/angles and intramolecular clashes	cleaner starting geometry for docking
Energy window	drops conformers more than *n* kcal/mol above the minimum	removes strained geometries
Post-optimisation RMS de-duplication (symmetry-aware heavy-atom RMSD)	minimisation often collapses conformers onto each other	keeps only distinct ones
Optional stereoisomer enumeration	expands unassigned stereocentres	avoids docking an arbitrary isomer

Other RDKit options not used here: Torsion-Fingerprint/Butina clustering for very large ensembles, and `rdDistGeom.EmbedMultipleConfs` with `useRandomCoords=True` (used automatically as a fallback when embedding fails). For a single rigid molecule set *Conformers to embed* = 1.

The output file is `<input>_confs.sdf` in the input folder.

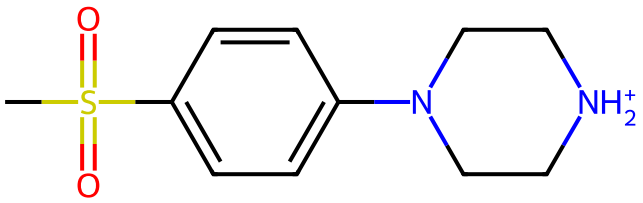
In [6]:
```
conf_panel = dt.ConformerPanel(input_mols, LIGAND_SDF)
display(conf_panel.widget())
if AUTO_RUN:
    conf_panel.run()
```

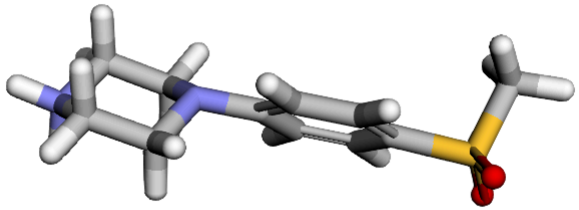
```
Wrote 33 conformers to example/Fordocking_confs.sdf
```

	name	embedded	kept	force field
0	FromXRay	10	4	MMFF94
1	sulphonamide	15	2	MMFF94
2	Phenyl	18	2	MMFF94
3	DimethylN	17	7	MMFF94
4	iProAmine	18	11	MMFF94
5	piperidine	14	2	MMFF94
6	amide	10	5	MMFF94

In [7]:
```python
# Check the conformer file (re-run this cell after generating)
CONF_SDF = conf_panel.out_path
if CONF_SDF.exists():
    confs = dt.load_sdf(CONF_SDF)
    print(f"{len(confs)} conformers in {CONF_SDF}")
    dt.molecule_browser(confs)
```

```
33 conformers in example/Fordocking_confs.sdf
1/33  FromXRay   heavy atoms: 16   SMILES: CS(=O)(=O)c1ccc(N2CC[NH2+]CC2)cc1
```





3. Protein and binding site

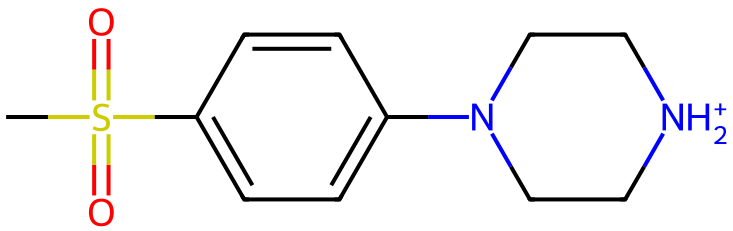
Two options:

- **Complex PDB** – a protein–ligand complex (e.g. `5RHD.pdb`). The HET groups are listed (waters excluded) and the largest is pre-selected; if the file contains several ligands choose the one that defines the binding site. The ligand is extracted (bond orders assigned from the hydrogens and formal charges, from an optional SMILES template, or by Open Babel when there are no hydrogens) and two files are written next to the PDB: `<stem>_protein.pdb` (protein, no ligand/other HETs) and `<stem>_ligand_<RES><chain><num>.pdb/.sdf`.
- **Prepared files** – as in the original notebook: a protein PDB with no ligand plus the ligand extracted from the binding site (PDB).

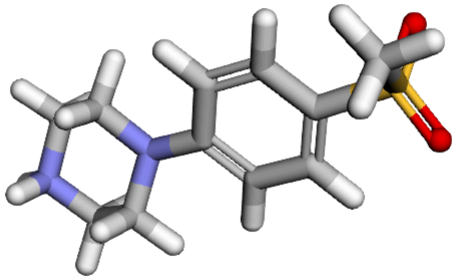
Press **Use this binding site** after changing anything.

In [8]:
```
site = dt.BindingSiteSetup(complex_pdb=Path("example/5RHD.pdb").resolve(),
                           protein_pdb=Path("example/proteinNoligand.pdb").resolve(),
                           ligand_pdb=Path("example/LigandOnly.pdb").resolve())
display(site.widget())
site.apply();   # applies the defaults; press the button to re-apply after changes
```

```
9 HET groups found (waters excluded); the largest is pre-selected - change it if needed.
Protein for docking : example/5RHD_protein.pdb  (4643 ATOM records)
Ligand (PDB)        : example/5RHD_ligand_LIGA408.pdb
Ligand (SDF)        : example/5RHD_ligand_LIGA408.sdf
Bond orders: RDKit DetermineBondOrders (net charge +1)
SMILES: CS(=O)(=O)c1ccc(N2CC[NH2+]CC2)cc1
```



LIG_A408



Images of the complex and the binding site

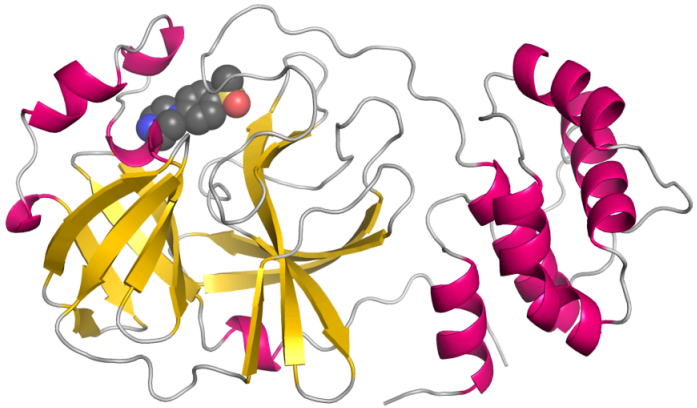
The protein is coloured by secondary structure with the **RasMol** scheme (helix magenta-red, sheet yellow, coil/turn white – drawn light grey so it shows on a white background); change `dt.PROTEIN_COLOURING` in the first code cell for rainbow or grey. Ray-traced with PyMOL and saved as `<stem>_complex.png` and `<stem>_site.png` next to the input structure. In the binding-site image residues within 5 Å of the ligand are shown as sticks, and polar contacts as orange dashes. Untick **Show binding-site side chains** to redraw the binding-site image without the side chains (and their polar contacts); the residues stay labelled.

In [9]:

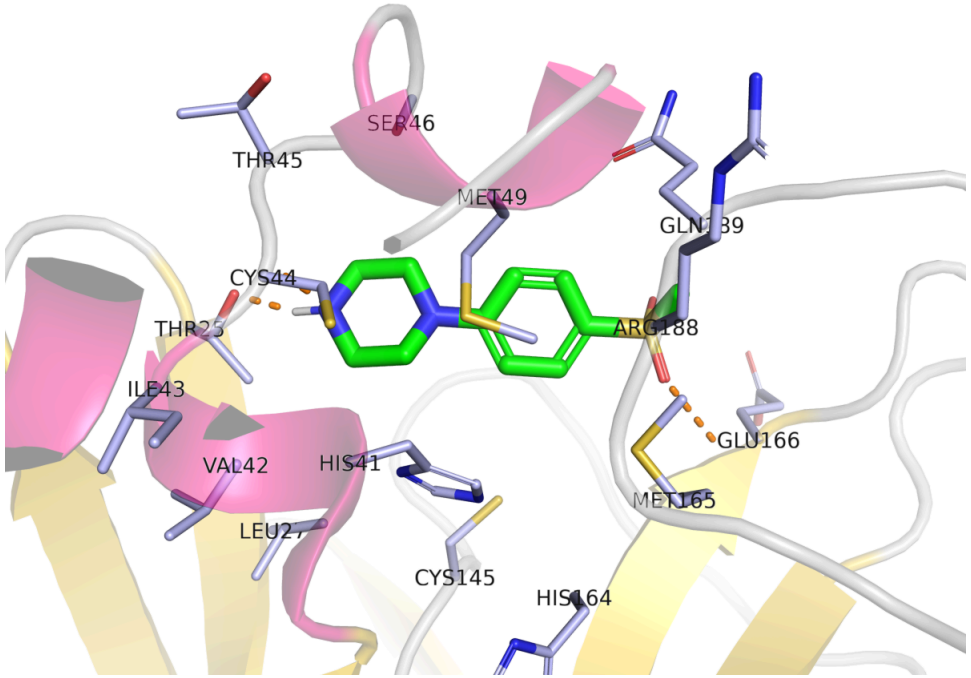
```
show_side_chains = dt.widgets.Checkbox(value=True, description="Show binding-site side chains",
                                       layout=dt.widgets.Layout(width="400px"))
image_out = dt.widgets.Output()

def _draw_images(_=None):
    global complex_png, site_png
    image_out.clear_output()
    with image_out:
        complex_png, site_png = dt.make_images(site.protein, site.ligand_ref, site.image_prefix,
                                               site_side_chains=show_side_chains.value)
        dt.show_images_side_by_side([complex_png, site_png],
                                    ["Protein with ligand bound", "Ligand in the binding site"])
        display(HTML(dt.colour_legend_html()))

show_side_chains.observe(_draw_images, names="value")
display(dt.widgets.VBox([show_side_chains, image_out]))
_draw_images()
```



Protein with ligand bound



Ligand in the binding site

Protein (RasMol secondary structure): helix sheet coil/turn

4. Docking with smina

The SARS-CoV-2 main protease (Mpro) cleaves the viral polyprotein and is essential for viral replication (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9283855/). The crystallised protein was the subject of a fragment screen at Diamond (https://www.diamond.ac.uk/industry/Case-Studies/Case-study-Fragment-Screening-to-fight-COVID-19.html); 5RHD is one of the resulting structures.

The panel is pre-filled with the settings from the original notebook (`--exhaustiveness 20 --seed 0 --autobox_ligand`) and `--cpu` set to the number of cores detected. Notes:

- **`--minimize`** performs only a local energy minimisation of the supplied coordinates (no search). Use it on poses that are already in the site, e.g. the crystal ligand SDF or a docked SDF — not on freshly generated conformers. The output defaults to `<input>_minimized.sdf` .
- **Docking mode** – *Rigid receptor* (default) or *Flexible side chains*. ⚠️ **Flexible docking is much slower** (typically 10–100× longer, rising steeply with the number of flexible residues). Residues can be chosen
  - **automatically** – every residue with an atom within a user-defined distance of the site-defining ligand (default 5 Å), or
  - **manually** – pick residues from the list (nearest first) and/or type extra ones (`A:41,A:145`).

  Gly, Ala and Pro are excluded (no side-chain torsions). smina then also writes `<output>_flexres.pdb` holding the flexed side chains for **every pose, in the same order as the docked SDF** – the viewer and AEV-PLIG use it to pair each ligand pose with its own receptor conformation.
- The docking log is written next to the output as `<output>.log` .

In [10]:
```
panel = dt.SminaPanel(receptor=site.protein, autobox_ligand=site.ligand_pdb, ligands=CONF_SDF)
display(panel.widget())
if AUTO_RUN:
    panel.run()
```

```
$ /usr/local/smina.osx.12 -r example/5RHD_protein.pdb -l example/Fordocking_confs.sdf -o
example/Fordocking_docked.sdf --seed 0 --cpu 24 --autobox_ligand example/5RHD_ligand_LIGA408.pdb --autobox_add 4.0 -
-exhaustiveness 20 --num_modes 9 --energy_range 3.0

  _______  _______ _________ _        _______
 (  ____ \(       )\__   __/( (    /|(  ___  )
 | (    \/| () () |   ) (   |  \  ( || (   ) |
 | (_____ | || || |   | |   |   \ | || (___) |
 (_____  )| |(_)| |   | |   | (\ \) ||  ___  |
       ) || |   | |   | |   | | \   || (   ) |
 /\____) || )   ( |___) (___| )  \  || )   ( |
 \_______)|/     \|\_______/|/    )_)|/     \|
smina is based off AutoDock Vina. Please cite appropriately.
Weights      Terms
-0.035579    gauss(o=0,_w=0.5,_c=8)
-0.005156    gauss(o=3,_w=2,_c=8)
0.840245     repulsion(o=0,_c=8)
-0.035069    hydrophobic(g=0.5,_b=1.5,_c=8)
-0.587439    non_dir_h_bond(g=-0.7,_b=0,_c=8)
1.923        num_tors_div
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.7       0.000      0.000
2       -5.6       3.048      3.573
3       -5.6       2.889      3.520
4       -5.5       3.154      3.475
5       -5.4       4.507      6.517
6       -5.2       5.647      7.298
7       -5.2       4.459      6.552
8       -5.2       2.974      3.394
9       -5.1       1.237      1.524
Refine time 0.895
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.7       0.000      0.000
2       -5.6       3.102      3.619
3       -5.6       2.356      2.753
4       -5.4       3.125      3.568
5       -5.4       2.059      2.209
6       -5.4       1.059      1.549
7       -5.4       4.526      6.619
8       -5.3       5.670      7.323
9       -5.2       4.472      6.555
Refine time 0.859
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.1       0.000      0.000
2       -6.1       0.181      1.214
3       -5.9       1.027      1.214
4       -5.8       0.972      1.653
5       -5.7       4.422      6.237
6       -5.5       1.396      1.513
7       -5.5       1.837      2.854
8       -5.4       1.588      2.112
9       -5.4       1.554      2.248
Refine time 0.908
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.1       0.000      0.000
2       -5.9       0.552      1.326
3       -5.7       1.741      2.392
4       -5.7       2.064      3.018
5       -5.6       1.489      1.645
6       -5.6       1.350      2.049
7       -5.6       1.753      2.696
8       -5.4       1.605      2.423
9       -5.4       4.094      6.097
Refine time 0.886
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.9       0.000      0.000
2       -5.9       4.832      6.692
3       -5.8       4.847      6.729
4       -5.8       3.698      3.921
5       -5.8       0.649      1.799
6       -5.7       4.188      6.553
7       -5.6       3.481      3.782
8       -5.6       1.101      1.712
9       -5.5       1.470      1.970
Refine time 0.938
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.9       0.000      0.000
2       -5.9       0.153      1.214
3       -5.7       3.757      4.046
4       -5.7       4.870      6.739
5       -5.6       4.236      6.534
6       -5.5       3.054      3.416
7       -5.5       4.883      6.705
8       -5.5       4.653      6.581
9       -5.4       4.375      6.080
Refine time 0.936
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.9       0.000      0.000
2       -6.9       0.236      1.509
3       -6.6       0.535      1.600
4       -6.6       4.085      7.428
5       -6.6       1.252      2.036
6       -6.6       4.724      6.989
7       -6.4       2.527      3.187
8       -6.2       4.154      7.643
9       -6.2       4.666      7.006
Refine time 1.578
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.9       0.000      0.000
2       -6.8       0.116      1.496
3       -6.6       0.571      1.900
4       -6.6       1.315      2.099
5       -6.5       4.913      6.990
6       -6.5       4.042      7.532
7       -6.5       2.571      3.247
8       -6.2       5.198      7.609
9       -6.2       3.991      7.100
Refine time 1.586
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.0       0.000      0.000
2       -5.9       2.593      3.088
3       -5.7       2.647      3.228
4       -5.7       1.277      1.846
5       -5.7       4.439      6.931
6       -5.5       1.560      2.126
7       -5.4       3.556      6.259
8       -5.4       2.017      2.676
9       -5.3       4.103      6.656
Refine time 1.193
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.9       0.000      0.000
2       -5.9       3.081      3.565
3       -5.8       3.085      3.873
4       -5.7       1.007      1.985
5       -5.5       4.524      6.999
6       -5.5       3.929      6.536
7       -5.4       4.012      6.606
8       -5.3       2.842      3.882
9       -5.3       2.937      7.023
Refine time 1.180
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.9       0.000      0.000
2       -5.8       2.711      3.263
3       -5.6       4.430      6.912
4       -5.5       1.075      1.462
5       -5.5       2.961      6.899
6       -5.4       3.719      6.524
7       -5.4       3.924      6.964
8       -5.4       3.568      6.289
9       -5.3       3.686      6.556
Refine time 1.199
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.3       0.000      0.000
2       -6.1       1.094      1.719
3       -5.9       3.062      6.561
4       -5.8       1.623      1.987
5       -5.8       1.608      2.216
6       -5.8       3.803      6.530
7       -5.8       2.968      6.587
8       -5.6       1.852      2.795
9       -5.5       1.499      1.711
Refine time 1.235
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.3       0.000      0.000
2       -6.2       0.575      1.315
3       -6.0       1.417      2.044
4       -5.9       1.329      1.841
5       -5.9       3.760      6.518
6       -5.8       2.059      3.003
7       -5.6       1.624      2.111
8       -5.6       1.669      2.545
9       -5.6       1.314      1.725
Refine time 1.248
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.8       0.000      0.000
2       -5.8       2.930      3.399
3       -5.7       4.404      7.229
4       -5.6       3.968      7.036
5       -5.5       3.826      6.976
6       -5.5       3.411      3.893
7       -5.4       3.082      3.726
8       -5.4       2.666      7.264
9       -5.3       1.200      2.081
Refine time 1.197
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.3       0.000      0.000
2       -6.1       1.153      1.795
3       -6.0       3.846      6.601
4       -5.9       3.102      6.504
5       -5.8       1.494      1.662
6       -5.8       1.470      1.840
7       -5.7       1.140      1.454
8       -5.7       2.904      6.567
9       -5.6       1.382      1.811
Refine time 1.221
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.1       0.000      0.000
2       -6.0       1.549      2.033
3       -6.0       0.913      1.467
4       -6.0       4.734      7.814
5       -6.0       4.659      7.791
6       -5.7       1.307      1.786
7       -5.7       4.595      5.947
8       -5.6       1.676      2.080
9       -5.6       3.289      4.835
Refine time 1.343
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.2       0.000      0.000
2       -6.1       0.535      1.224
3       -6.0       1.522      1.973
4       -5.9       4.672      7.793
5       -5.9       4.655      7.806
6       -5.8       4.737      7.898
7       -5.6       3.833      5.098
8       -5.5       2.462      2.938
9       -5.5       2.987      3.525
Refine time 1.215
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.1       0.000      0.000
2       -5.9       4.691      7.723
3       -5.9       4.750      7.791
4       -5.8       1.447      1.752
5       -5.7       4.736      7.675
6       -5.7       4.626      7.720
7       -5.7       3.466      4.669
8       -5.6       3.010      3.765
9       -5.6       3.202      4.421
Refine time 1.278
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.1       0.000      0.000
2       -6.1       3.994      4.715
3       -6.0       3.911      4.670
4       -5.9       5.939      8.156
5       -5.9       3.663      4.437
6       -5.8       5.979      8.208
7       -5.8       3.667      4.805
8       -5.7       5.891      7.919
9       -5.7       4.466      6.838
Refine time 1.341
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.2       0.000      0.000
2       -6.1       3.295      4.287
3       -6.1       3.252      4.335
4       -6.0       3.489      4.566
5       -5.9       3.105      4.015
6       -5.8       5.704      7.935
7       -5.8       2.539      3.870
8       -5.8       2.883      3.979
9       -5.8       3.394      4.212
Refine time 1.254
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.2       0.000      0.000
2       -6.2       0.105      1.106
3       -6.0       1.451      1.580
4       -5.9       3.431      4.499
5       -5.9       4.730      7.744
6       -5.8       4.794      7.721
7       -5.8       4.738      7.717
8       -5.8       3.148      4.672
9       -5.7       3.406      4.538
Refine time 1.301
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.1       0.000      0.000
2       -6.1       1.067      1.072
3       -6.0       4.917      7.977
4       -5.9       4.742      7.755
5       -5.9       4.932      7.910
6       -5.9       4.747      7.786
7       -5.8       1.055      2.010
8       -5.7       1.122      1.391
9       -5.5       3.016      3.323
Refine time 1.330
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.1       0.000      0.000
2       -6.0       1.128      1.434
3       -5.9       4.905      7.975
4       -5.9       4.810      7.802
5       -5.8       4.882      7.900
6       -5.7       4.890      7.976
7       -5.6       3.493      4.803
8       -5.6       1.141      1.931
9       -5.5       2.671      3.278
Refine time 1.409
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.5       0.000      0.000
2       -6.4       4.879      6.273
3       -6.4       4.236      6.418
4       -6.4       1.348      1.802
5       -6.4       1.924      2.491
6       -6.1       4.068      6.808
7       -6.1       4.138      7.214
8       -6.1       2.800      4.128
9       -6.1       1.802      2.298
Refine time 1.397
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.1       0.000      0.000
2       -6.0       3.466      4.377
3       -6.0       2.772      4.034
4       -5.9       2.881      3.898
5       -5.8       2.905      4.058
6       -5.8       5.730      7.967
7       -5.8       2.891      3.839
8       -5.8       2.653      3.789
9       -5.7       4.464      6.922
Refine time 1.289
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.2       0.000      0.000
2       -6.0       1.494      1.762
3       -5.9       4.793      7.726
4       -5.9       4.794      7.823
5       -5.8       1.064      1.370
6       -5.8       4.767      7.743
7       -5.8       3.005      3.572
8       -5.7       3.493      4.989
9       -5.7       4.619      5.396
Refine time 1.282
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.9       0.000      0.000
2       -5.8       4.962      6.724
3       -5.7       4.985      6.594
4       -5.7       5.127      6.891
5       -5.6       0.935      1.504
6       -5.5       4.661      6.628
7       -5.4       1.200      1.934
8       -5.4       4.688      6.490
9       -5.4       4.986      6.738
Refine time 0.864
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -6.0       0.000      0.000
2       -5.9       1.677      2.201
3       -5.9       2.486      2.805
4       -5.8       1.846      2.127
5       -5.8       2.415      2.964
6       -5.8       1.301      1.932
7       -5.6       1.440      1.733
8       -5.6       4.946      7.038
9       -5.3       4.286      6.661
Refine time 0.905
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.7       0.000      0.000
2       -5.7       4.423      6.917
3       -5.7       2.773      3.033
4       -5.7       4.198      4.633
5       -5.6       4.370      7.001
6       -5.5       4.206      6.885
7       -5.4       3.652      6.185
8       -5.3       1.833      2.660
9       -5.3       3.621      4.249
Refine time 0.882
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.7       0.000      0.000
2       -5.7       1.527      2.410
3       -5.7       2.608      3.104
4       -5.7       3.381      5.972
5       -5.6       4.180      6.359
6       -5.6       3.477      6.007
7       -5.5       2.496      3.749
8       -5.5       2.521      3.669
9       -5.4       1.802      2.307
Refine time 0.849
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.8       0.000      0.000
2       -5.7       0.633      1.341
3       -5.6       2.893      6.166
4       -5.6       2.882      6.049
5       -5.5       2.739      6.243
6       -5.4       4.406      6.589
7       -5.3       5.229      6.045
8       -5.3       4.325      6.379
9       -5.3       3.236      4.039
Refine time 0.820
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
```

```
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.7       0.000      0.000
2       -5.7       2.838      3.864
3       -5.7       1.604      2.490
4       -5.7       2.770      3.128
5       -5.7       3.383      5.990
6       -5.7       2.550      3.054
7       -5.6       4.165      6.434
8       -5.5       2.197      3.394
9       -5.4       4.082      6.143
Refine time 0.872
Using random seed: 0
0%   10   20   30   40   50   60   70   80   90   100%
|----|----|----|----|----|----|----|----|----|----|
     | (kcal/mol) | rmsd l.b.| rmsd u.b.
1       -5.7       0.000      0.000
2       -5.7       0.408      1.300
3       -5.6       2.748      6.207
4       -5.5       0.818      1.897
5       -5.5       2.737      6.079
6       -5.5       5.226      5.992
7       -5.4       4.577      6.949
8       -5.3       2.163      2.614
9       -5.3       2.899      3.631
Refine time 0.836
Loop time 42.533

smina finished with exit code 0. Output: example/Fordocking_docked.sdf
Log: example/Fordocking_docked.log
```

5. View docking results

Poses are sorted by smina score (best first). Step through them with the slider; the ligand from the original PDB (reference) is shown with red carbons and the current pose with green carbons. Untick **Ligand from PDB (red)** to hide the reference ligand.

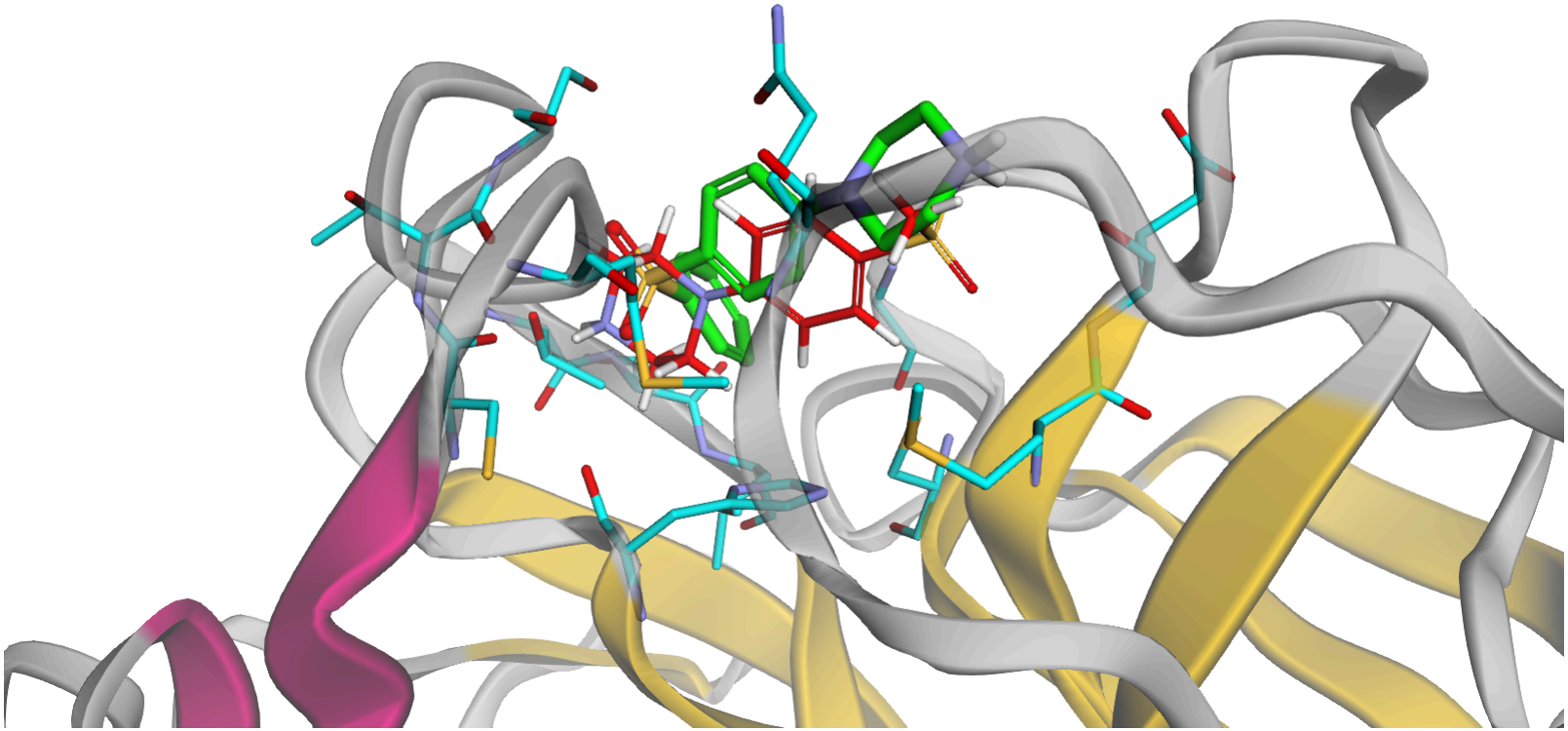
After **flexible docking** each pose is shown together with *its own* flexed side chains (magenta carbons), with the original crystal side-chain positions overlaid in thin grey (toggle). The receptor for each pose is rebuilt from the rigid protein plus that pose's model in `<output>_flexres.pdb`.

In [11]:
```
# Re-run this cell after smina has finished
DOCKED_SDF = Path(panel.output.value)
poses, pose_table, FLEX_PDB = [], None, None
if DOCKED_SDF.exists():
    poses = dt.load_poses(DOCKED_SDF)
    FLEX_PDB = dt.flex_file_for(DOCKED_SDF)       # present only after flexible docking
    FLEX_PDB = FLEX_PDB if FLEX_PDB.exists() else None
    print(f"{len(poses)} poses in {DOCKED_SDF}")
    if FLEX_PDB:
        print(f"Flexible side chains: {FLEX_PDB}")
    pose_table = dt.results_browser(poses, site.protein, site.ligand_ref, flex_pdb=FLEX_PDB)
else:
    print(f"{DOCKED_SDF} not found - press 'Run smina' above, then re-run this cell")
```

```
297 poses in example/Fordocking_docked.sdf
```

1/297 **Phenyl** (pose_63) smina affinity: **-6.91** kcal/mol

Protein (RasMol secondary structure): helix sheet coil/turn



In [12]:
```
pose_table.head(10) if pose_table is not None else None
```

Out[12]:

	pose_id	name	minimizedAffinity
0	pose_63	Phenyl	-6.90800
1	pose_54	Phenyl	-6.88005
2	pose_55	Phenyl	-6.86180
3	pose_64	Phenyl	-6.81614
4	pose_65	Phenyl	-6.64477
5	pose_66	Phenyl	-6.62420
6	pose_56	Phenyl	-6.61338
7	pose_57	Phenyl	-6.59131
8	pose_58	Phenyl	-6.57097
9	pose_59	Phenyl	-6.56943

6. Binding-affinity prediction with AEV-PLIG

AEV-PLIG is a graph neural network scoring function (atomic environment vectors + protein–ligand interaction graphs). The model used is `model_GATv2Net_ligsim90_fep_benchmark` (ensemble of 10), copied with the prediction code into `aev_plig/` in this project. Predictions are **pK** values (higher = tighter binding); the spread across the 10 models is reported as `AEV_PLIG_pK_std`.

All results are exported to `predictions/` next to the docked SDF:

File	Content
`poses/<docked>/pose_<i>.sdf`	one file per pose (AEV-PLIG input)
`poses/<docked>/pose_<i>_receptor.pdb`	flexible docking only: the receptor with that pose's flexed side chains (each pose is scored against its own receptor)
`<docked>_aevplig_input.csv`	AEV-PLIG input table
`<docked>_predictions.csv`	raw AEV-PLIG output (10 models + mean)
`<docked>_summary.csv`	name, pose, smina score, pK, pK std, best-pose flags
`<docked>_scored.sdf`	all poses annotated with pK
`<docked>_best_poses.sdf`	highest-pK pose per ligand
`<docked>_best_poses_table.csv`	best pose per ligand **plus the ligand from the original PDB** in its crystal pose: AEV-PLIG pK (also the red dashed line on the plot) and smina scores – `minimizedAffinity` after local minimisation in the site (comparable with the docked poses), `smina_score_only` for the unmodified crystal coordinates and `minimizedRMSD` (how far minimisation moved it)
`pdb_ligand/`	the PDB ligand as scored by smina, its minimised pose and the smina log
`<docked>_best_poses.pse`	PyMOL session of the best poses (section 7)
`best_poses_pdb/<docked>/<ligand>_<pose>_ligand.pdb` and `..._complex.pdb`	highest-pK pose per ligand in PDB format: ligand only, and protein–ligand complex (ligand = chain L, resname LIG) using that pose's own receptor (flexed side chains after flexible docking); REMARKs give smina score and pK

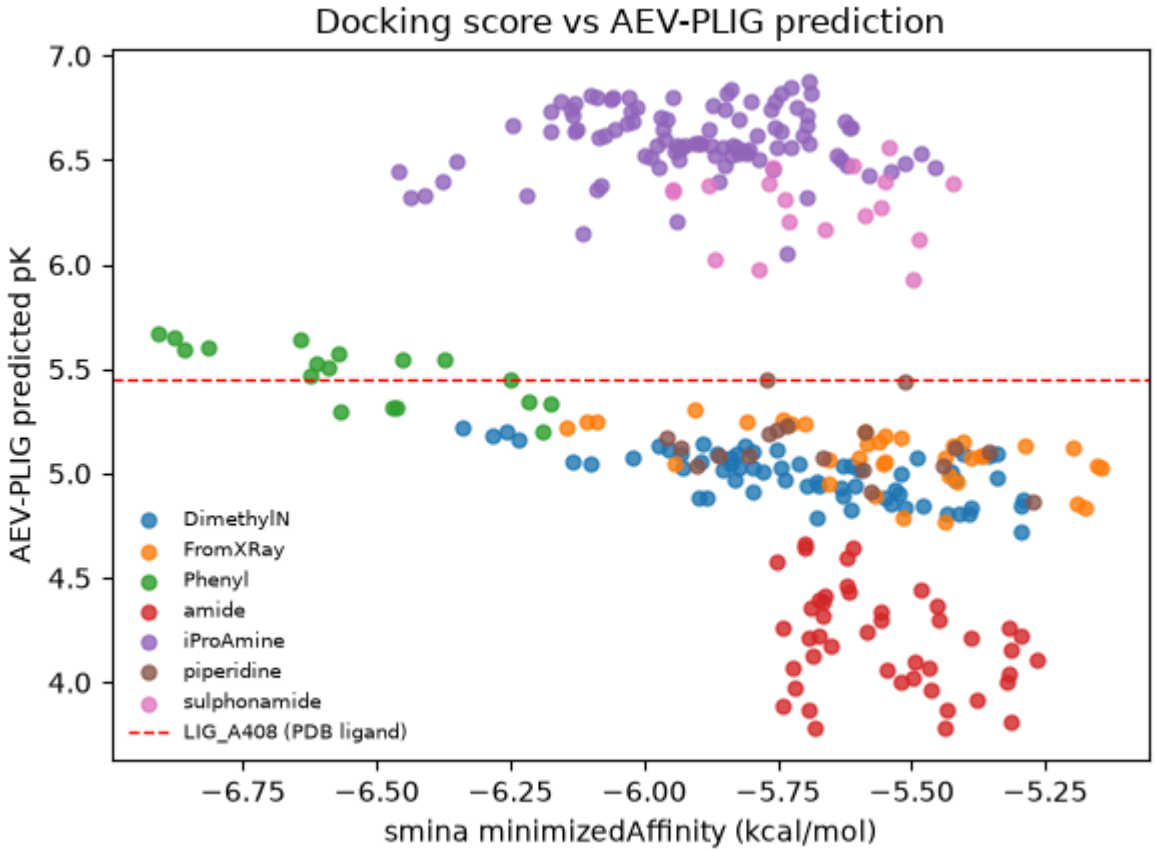
File	Content
`<docked>_smina_vs_aevplig.png`	score comparison plot
copies of docked SDF, receptor, `_flexres.pdb` (flexible), reference ligand, conformers and images	for a self-contained record

In [13]:
```
predict_button = dt.widgets.Button(description="Run AEV-PLIG", button_style="success", icon="play")
predict_out = dt.widgets.Output()
summary = None

def _predict(_=None):
    global summary
    predict_out.clear_output()
    with predict_out:
        if not DOCKED_SDF.exists():
            print("No docked SDF yet - run smina and the 'View docking results' cell first")
            return
        extras = [CONF_SDF, site.ligand_pdb, site.ligand_ref, complex_png, site_png,
                  DOCKED_SDF.with_suffix(".log")]
        # smina scores for the ligand from the PDB in its crystal pose (same smina + scoring function)
        ref_smina = None
        if site.ligand_mol is not None:
            ref_smina = dt.smina_score_ligand(site.ligand_mol, site.protein,
                                              DOCKED_SDF.parent / "predictions" / "pdb_ligand",
                                              smina=panel.smina.value, scoring=panel.scoring_arg)
        summary = dt.run_aev_plig(DOCKED_SDF, site.protein, extra_files=extras, flex_pdb=FLEX_PDB,
                                  reference_mol=site.ligand_mol,     # also score the ligand from the PDB
                                  reference_smina=ref_smina)
        display(summary.head(10))

predict_button.on_click(_predict)
display(dt.widgets.VBox([predict_button, predict_out]))
if AUTO_RUN:
    _predict()
```

```
[11:57:07] Warning: molecule is tagged as 2D, but at least one Z coordinate is not zero. Marking the mol as 3D.
PDB ligand smina: score_only -5.05, minimized -5.84 kcal/mol (RMSD 0.42 Å)
$ python process_and_predict.py --dataset_csv=example/predictions/Fordocking_docked_aevplig_input.csv --
data_name=Fordocking_docked --trained_model_name=model_GATv2Net_ligsim90_fep_benchmark --num_workers=24 --
output_dir=example/predictions
PDB ligand LIG_A408: AEV-PLIG pK 5.45 +/- 0.44
Best pose per ligand exported as PDB (7 ligands) to example/predictions/best_poses_pdb/Fordocking_docked
```




```
All results exported to example/predictions
```

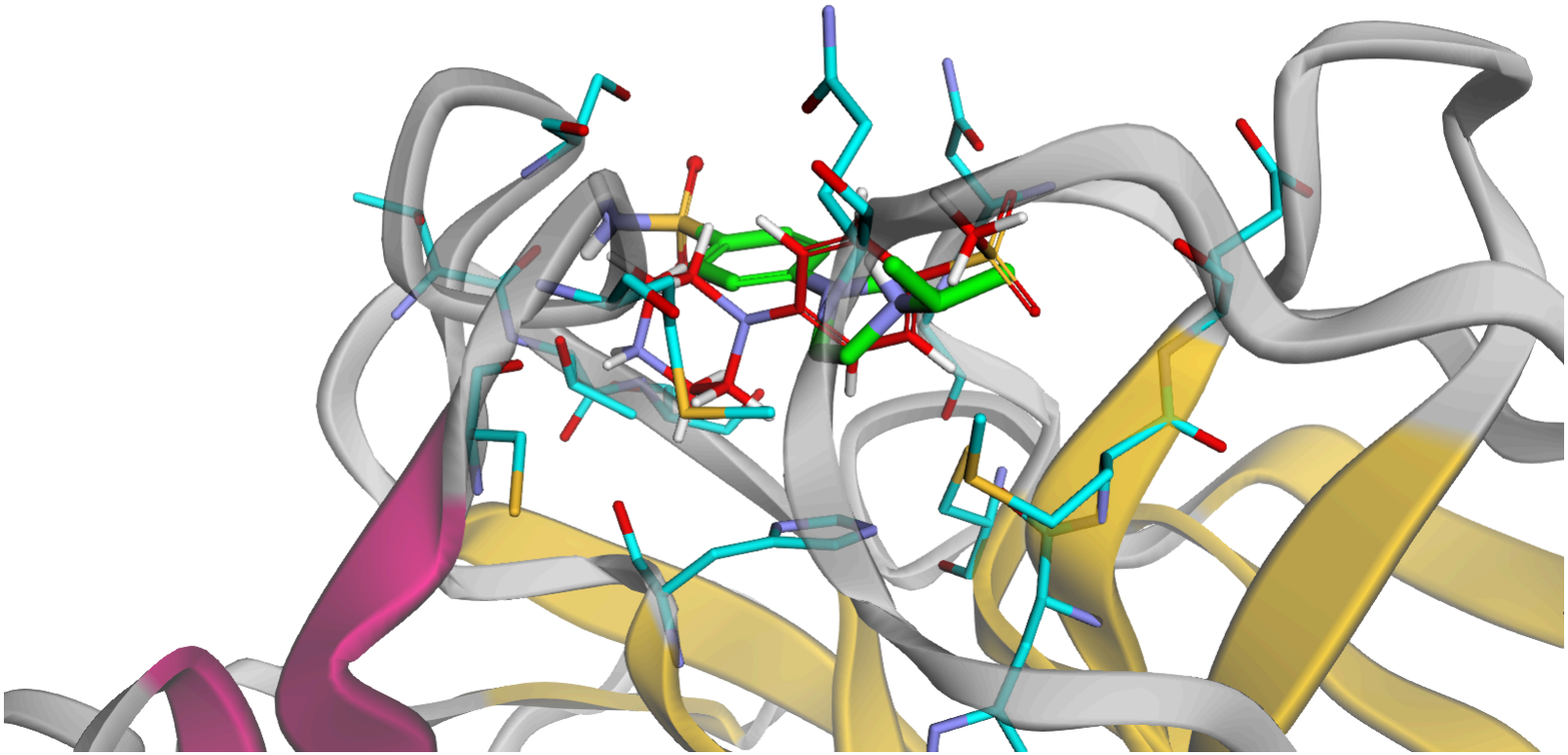
	pose_id	name	minimizedAffinity	AEV_PLIG_pK	AEV_PLIG_pK_std	best_smina_for_ligand	best_aevplig_for_ligand	rec
0	pose_159	iProAmine	-5.69195	6.873839	0.481758	False	True	5RHD_p
1	pose_224	iProAmine	-5.72727	6.847566	0.387764	False	False	5RHD_p
2	pose_229	iProAmine	-5.83847	6.837327	0.507700	False	False	5RHD_p
3	pose_170	iProAmine	-5.68985	6.823840	0.389973	False	False	5RHD_p
4	pose_220	iProAmine	-5.84620	6.822902	0.481887	False	False	5RHD_p
5	pose_188	iProAmine	-5.74313	6.817914	0.455671	False	False	5RHD_p
6	pose_163	iProAmine	-6.09893	6.808291	0.431737	False	False	5RHD_p
7	pose_164	iProAmine	-6.02817	6.805348	0.408655	False	False	5RHD_p
8	pose_173	iProAmine	-6.08871	6.801361	0.451409	False	False	5RHD_p
9	pose_153	iProAmine	-6.05932	6.800661	0.404848	False	False	5RHD_p

Step through poses ranked by AEV-PLIG pK

In [14]:
```
if summary is not None:
    dt.results_browser(poses, site.protein, site.ligand_ref, predictions=summary, sort_by="AEV_PLIG_pK",
                       flex_pdb=FLEX_PDB)
```

1/297 **iProAmine** (pose_159) smina affinity: **-5.69** kcal/mol AEV-PLIG pK: **6.87**

Protein (RasMol secondary structure): helix sheet coil/turn



In [15]:
```
# Best pose per ligand by AEV-PLIG, plus the ligand from the original PDB in its crystal pose
# (best poses are also exported as PDB: ligand only + protein-ligand complex)
if summary is not None:
    best_table = pd.read_csv(DOCKED_SDF.parent / "predictions" / f"{DOCKED_SDF.stem}_best_poses_table.csv")
    display(best_table.style.format({"minimizedAffinity": "{:.2f}", "AEV_PLIG_pK": "{:.2f}",
                                     "AEV_PLIG_pK_std": "{:.2f}", "smina_score_only": "{:.2f}",
                                     "minimizedRMSD": "{:.2f}"}, na_rep="–")
            .apply(lambda r: ["background-color: #fde2e2" if r["source"] == "ligand from PDB" else ""] * len(r),
                   axis=1).hide(axis="index"))
    best_pdb_dir = DOCKED_SDF.parent / "predictions" / "best_poses_pdb" / DOCKED_SDF.stem
    print(f"PDB files in {best_pdb_dir}:")
    for f in sorted(best_pdb_dir.glob("*.pdb")):
        print("  ", f.name)
```

source	name	pose_id	minimizedAffinity	AEV_PLIG_pK	AEV_PLIG_pK_std	smina_score_only	minimizedRMSD
docked (best AEV-PLIG pose)	iProAmine	pose_159	-5.69	6.87	0.48	–	–
docked (best AEV-PLIG pose)	sulphonamide	pose_44	-5.54	6.57	0.55	–	–
docked (best AEV-PLIG pose)	Phenyl	pose_63	-6.91	5.67	0.50	–	–
docked (best AEV-PLIG pose)	piperidine	pose_247	-5.77	5.45	0.36	–	–
docked (best AEV-PLIG pose)	FromXRay	pose_20	-5.91	5.31	0.26	–	–
docked (best AEV-PLIG pose)	DimethylN	pose_126	-6.34	5.22	0.34	–	–
docked (best AEV-PLIG pose)	amide	pose_288	-5.70	4.67	0.48	–	–
ligand from PDB	LIG_A408	crystal pose	-5.84	5.45	0.44	-5.05	0.42

```
PDB files in example/predictions/best_poses_pdb/Fordocking_docked:
   DimethylN_pose_126_complex.pdb
   DimethylN_pose_126_ligand.pdb
   FromXRay_pose_20_complex.pdb
   FromXRay_pose_20_ligand.pdb
   Phenyl_pose_63_complex.pdb
   Phenyl_pose_63_ligand.pdb
   amide_pose_288_complex.pdb
   amide_pose_288_ligand.pdb
   iProAmine_pose_159_complex.pdb
   iProAmine_pose_159_ligand.pdb
   piperidine_pose_247_complex.pdb
   piperidine_pose_247_ligand.pdb
   sulphonamide_pose_44_complex.pdb
   sulphonamide_pose_44_ligand.pdb
```

7. Export best poses as a PyMOL session

Saves `predictions/<docked>_best_poses.pse` with the best AEV-PLIG pose of each ligand, ready to open in PyMOL:

- `receptor` – cartoon (RasMol secondary-structure colours) with binding-site side chains as light-blue sticks
- one group per ligand, ranked by pK (`01_<ligand>`, `02_<ligand>` …) containing the pose (`_pose`), its polar contacts (`_hbonds`) and, after flexible docking, that pose's flexed side chains (`_flexed`, magenta)
- `PDB_ligand` – the ligand from the original PDB (red carbons)

Only the top-ranked ligand is switched on initially (tick **Show all ligands** to enable every one); toggle groups in PyMOL's object panel. Each pose's title shows its name, pose id, pK and smina score.

In [16]:
```
pse_all = dt.widgets.Checkbox(value=False, description="Show all ligands when the session opens")
pse_button = dt.widgets.Button(description="Export PyMOL session", button_style="success", icon="download")
pse_out = dt.widgets.Output()

def _export_pse(_=None):
    pse_out.clear_output()
    with pse_out:
        if summary is None:
            print("Run AEV-PLIG first (section 6)")
            return
        dt.export_pymol_session(DOCKED_SDF, site.protein, reference_ligand=site.ligand_ref,
                                enable="all" if pse_all.value else "top")

pse_button.on_click(_export_pse)
display(dt.widgets.VBox([pse_all, pse_button, pse_out]))
if AUTO_RUN:
    _export_pse()
```

```
PyMOL session with 7 best poses written to example/predictions/Fordocking_docked_best_poses.pse
```